

# PG Management SaaS

## Product Requirements Document

**Version:** 1.0

**Product Type:** Multi-tenant SaaS

**Primary Platforms:** Web application first, mobile-friendly resident experience

**Target Users:** PG owners, PG managers/wardens, residents

---

## 1. Product Overview

The PG Management SaaS is a property and resident management platform designed for Paying Guest accommodations.

The platform will allow PG owners to manage one or multiple PG properties from a single account while keeping each PG's operations, residents, finances, rooms, billing, utilities and settings separately configurable.

The system will manage the complete resident lifecycle:

**Reservation → Bed Allocation → Move-In → Rent & Utility Billing → Payments → Complaints/Food → Transfers → Notice Period → Checkout → Final Settlement**

The product will be built as a SaaS platform so different PG businesses can independently use the system without accessing each other's data.

---

## 2. Problem Statement

Most PG businesses currently manage their operations through combinations of:

- Registers
- Excel sheets
- WhatsApp
- UPI screenshots
- Manual rent reminders
- Paper receipts
- Manual electricity calculations
- Separate food polls
- Manual room allocation

This creates problems such as:

- Difficulty tracking vacant beds
- Duplicate room/bed allocation
- Missing rent payments
- No proper payment history
- Difficulty calculating electricity
- Poor security-deposit tracking
- No proper resident history
- Difficulty managing multiple PGs
- No structured complaint system
- Manual food voting
- Lack of centralized documents
- Lack of transaction audit history

The application will centralize all of these operations.

---

### 3. Product Vision

Build a PG operating system where an owner or manager can immediately answer:

- Who is currently staying in the PG?
- Which beds are vacant?
- Which beds are reserved?
- Who has rent due?
- How much money has been collected?
- How much security deposit is held?
- Which residents have pending electricity bills?
- Which residents have complaints?
- What food has residents selected?
- Who is leaving soon?
- Which rooms are becoming available?

For residents, the application should provide a simple self-service experience for:

- Payments
  - Receipts
  - Bills
  - Food voting
  - Complaints
  - Documents
  - Notices
  - Profile information
-

## 4. Product Hierarchy

The system will follow this hierarchy:

```
SaaS Platform
  ↓
Organization / Owner
  ↓
PG Property
  ↓
Floor
  ↓
Room
  ↓
Bed
  ↓
Resident Allocation
```

Example:

```
ABC Living
|
|— PG Noida
|   |
|   |— Floor 1
|   |   |
|   |   |— Room 101
|   |   |   |
|   |   |   |— 101-A
|   |   |   |— 101-B
|   |   |— Room 102
|   |— Floor 2
|— PG Gurgaon
```

One owner may operate multiple PG properties.

---

## 5. User Roles

The V1 application will support three primary roles.

### 5.1 Owner / Admin

The Owner/Admin has the highest organizational access.

Owner can:

- Create and manage PG properties
- Create floors
- Create rooms
- Configure room types
- Configure beds
- Configure default rent
- Manage managers
- View residents
- View occupancy
- View payments
- View dues
- View security deposits
- Configure electricity billing
- Configure PG policies
- Configure notice period
- Configure lock-in period
- Configure available add-ons
- View reports
- View financial dashboards
- View complaints
- Configure food services

The owner will not see resident food voting results where this restriction is configured.

---

## 5.2 Manager / Warden

Manager handles daily PG operations.

Manager can:

- Add residents
- Reserve beds
- Allocate beds
- Record security deposits
- Complete move-in
- Manage resident details
- Upload resident documents
- Record rent payments
- Record partial payments
- Generate receipts
- Record electricity meter readings
- Calculate electricity bills
- Manage resident services/add-ons
- Manage food menu

- View food poll results
- Manage complaints
- Publish notices
- Manage room transfers
- Manage PG-to-PG transfers
- Process notice period
- Process move-out
- Perform final settlement
- Create payment corrections/reversals

Manager cannot permanently edit/delete finalized financial transactions.

---

## 5.3 Resident

Resident can:

- View room and bed information
- View current rent
- View bills
- View outstanding dues
- View electricity charges
- View payment history
- Download receipts
- View security-deposit information
- View food menu
- Participate in food polls
- Submit complaints
- Track complaint status
- View notices
- Receive rent notifications
- Upload allowed documents
- Edit permitted profile information
- View current add-on services

Residents cannot change financial amounts.

---

## 6. Multi-Tenant SaaS Requirement

The system must support multiple independent PG businesses.

Example:

Owner A  
├ PG 1  
└ PG 2

Owner B  
├ PG 1  
├ PG 2  
└ PG 3

Owner A must never be able to access Owner B's:

- Residents
- Rooms
- Payments
- Documents
- Reports
- Managers
- Settings

Tenant isolation must be enforced at backend/data-access level and not only by hiding UI elements.

---

## 7. PG Configuration

Each PG must maintain independent configuration.

Configuration should include:

- PG name
- Address
- Contact details
- Number of floors
- Electricity calculation method
- Electricity unit rate
- Food configuration
- Notice period
- Lock-in period
- Security-deposit policy
- Reservation cancellation policy
- Mandatory KYC documents
- Optional KYC documents
- Available resident add-ons
- Receipt details
- Payment methods
- Rent reminder settings

---

## 8. Floor Management

Owner or manager can create multiple floors.

Each floor can have a different number of rooms.

Example:

Ground Floor

6 rooms

First Floor

10 rooms

Second Floor

8 rooms

Floor information should include:

- PG ID
- Floor name
- Floor number
- Number of rooms
- Electricity meter information if applicable
- Active/inactive status

---

## 9. Room Management

Every room belongs to a floor.

Supported room configurations include:

- Private room
- Single occupancy
- Double sharing
- Triple sharing
- Four sharing
- Custom type

Room type should define a default rent.

Example:

Private Room  
Default Rent: ₹20,000

Double Sharing  
Default Per-Bed Rent: ₹12,000

Triple Sharing  
Default Per-Bed Rent: ₹10,000

The default rent acts only as a starting value.

The final agreed rent is stored against the resident's stay.

---

## 10. Bed Management

Each room may contain one or multiple beds.

Example:

Room 101

101-A  
101-B  
101-C

Each bed can have one of the following internal states:

- Available
- Reserved
- Occupied
- Maintenance / unavailable

A bed cannot simultaneously be assigned to more than one active resident.

---

## 11. Resident Management

Resident information should include:

## Basic Information

- Full name
- Mobile number
- Email
- Date of birth
- Gender
- Profile photo

## Address Information

- Permanent address
- Current address if applicable

## Emergency Information

- Emergency contact name
- Emergency contact number
- Relationship

## Professional / Educational Information

- Occupation
  - Company name
  - College/institution name
- 

# 12. Resident KYC

KYC requirements must be configurable per PG.

Supported documents include:

- Aadhaar
- Driving Licence
- Passport
- PAN
- Company ID
- College ID
- Resident photo
- Other PG-defined document

Example:

PG A:

Aadhaar – Mandatory  
Driving Licence – Mandatory  
Company ID – Optional

PG B:

Aadhaar – Mandatory  
College ID – Mandatory  
Driving Licence – Optional

Documents may be uploaded as:

- Image
- PDF

Sensitive documents should not be shown unnecessarily throughout the application.

Normal resident listings should display verification status instead of complete document numbers.

---

## 13. Reservation Flow

A prospective resident may reserve a bed before moving into the PG.

Example:

Bed: 101-B  
Reservation Date: 10 September  
Expected Move-In: 15 September  
Deposit Received: ₹5,000

Once reserved:

Available → Reserved

Rent must NOT start from reservation date.

Rent starts only from actual move-in date.

Reservation cancellation is manually managed by the PG.

Automatic reservation expiry is not required in V1.

---

## 14. Security Deposit

Security deposit must be tracked independently from rent.

PGs can configure deposit policies.

Examples:

- Full one-month rent
- Half-month rent
- Fixed deposit amount
- Custom amount

Security deposit may be paid partially.

Example:

Required Deposit: ₹20,000

Payment 1: ₹5,000 UPI

Payment 2: ₹10,000 Cash

Payment 3: ₹5,000 UPI

Total Received: ₹20,000

Pending: ₹0

The default reservation amount is non-refundable, but refund policies can be configured by the PG.

---

## 15. Move-In

Actual rent starts from the resident's move-in date.

On move-in:

Reserved Bed → Occupied

The system creates a Resident Stay.

The Resident Stay should contain:

- Resident
  - PG
  - Floor
  - Room
  - Bed
  - Move-in date
  - Agreed rent
  - Rent billing cycle
  - Security deposit
  - Lock-in details
  - Notice-period rules
  - Add-on services
- 

## 16. Resident Stay

Resident identity must be separate from stay history.

A resident may stay in different rooms or PGs without creating a new resident record.

Example:

Resident: Rahul

Stay 1  
PG Noida  
101-A  
01 Jan – 15 Mar

Stay 2  
PG Noida  
205-B  
15 Mar – 01 Jul

Stay 3  
PG Gurgaon  
304-A  
01 Jul – Current

This preserves resident history.

---

## 17. Rent Configuration

Rent is based primarily on room/bed type but can differ per resident.

Example:

Triple Sharing Default Rent: ₹10,000

Rahul Agreed Rent: ₹9,500

Aman Agreed Rent: ₹10,000

Final agreed rent must be stored against the resident's stay.

---

## 18. Individual Rent Cycle

Each resident can have their own rent billing cycle.

Example:

Move-in date:

17 September

Billing cycle:

17 Sep – 16 Oct

17 Oct – 16 Nov

17 Nov – 16 Dec

The system must not assume that rent is always due on the first day of the month.

---

## 19. Month-End Handling

If a resident's rent date falls on the 31st and a month has fewer days, the billing date will become the last valid date of that month.

Example:

31 January  
28 February  
31 March  
30 April

---

## 20. Rent Revision

Managers/owners can revise rent with a future effective date.

Example:

Current Rent: ₹10,000  
  
New Rent: ₹11,000  
Effective From: 1 April

Historical rent values must not be overwritten.

The system must preserve rent history.

---

## 21. Room Transfer

Resident may move to another room or bed.

Example:

101-A  
₹10,000/month  
  
↓  
  
Private Room 205  
₹20,000/month

If transfer occurs in the middle of a billing cycle, the system should calculate prorated rent.

Manager can override the generated calculation.

The overridden value must be recorded with:

- Original calculated amount
  - Final amount
  - Changed by
  - Reason
  - Date
- 

## 22. PG-to-PG Transfer

A resident can transfer between PG properties owned by the same organization.

Resident identity and KYC remain the same.

A new stay/allocation record is created.

Previous:

PG Noida  
Room 101-A

New:

PG Gurgaon  
Room 304-B

Historical data must remain accessible.

---

## 23. Lock-In Period

Each PG may define its own lock-in period.

Example:

Lock-In Period: 3 months

If resident moves in on:

1 January

Normal lock-in ends after three months.

The system should warn manager/resident if checkout or transfer is initiated during lock-in.

Authorized manager/admin may override the restriction.

Lock-in period may be disabled for some PGs.

---

## 24. Notice Period

Each PG can define a notice period.

Example:

Notice Period: 30 days

Notice period should be configurable.

Residents/manager should be able to record:

- Notice submitted date
  - Planned checkout date
  - Approved checkout date
- 

## 25. Resident Add-On Services

Additional services should not be permanently included in base rent.

PG can define services such as:

- Laundry
- AC
- Parking
- Food
- Extra meal plan
- Housekeeping
- Storage
- Other custom services

Example:

|           |        |
|-----------|--------|
| Base Rent | ₹9,000 |
| Laundry   | ₹500   |
| Parking   | ₹800   |

If resident does not use laundry:

Laundry charge = ₹0

Add-ons can be enabled/disabled per resident.

---

## 26. Monthly Billing

The billing system should combine the resident's applicable charges.

Example:

|                      |         |
|----------------------|---------|
| Base Rent            | ₹10,000 |
| Laundry              | ₹500    |
| Electricity          | ₹750    |
| Food Add-On          | ₹2,000  |
| Previous Outstanding | ₹500    |
| Total Due            | ₹13,750 |

The billing system should generate an invoice for the relevant cycle.

---

## 27. Payment Management

Supported payment modes should include:

- Cash
- UPI
- Bank transfer
- Card
- Other

Partial payments must be supported.

Example:

Invoice:

₹20,000

Payments:

₹14,000 UPI  
₹6,000 Cash

Balance:

₹0

Each payment should store:

- Amount
- Payment date
- Payment mode
- Reference number
- Invoice
- Recorded by
- Optional payment proof
- Notes

---

## 28. Payment Ledger

Financial data must follow a ledger-style model.

Do not simply store:

Paid = Yes / No

Example ledger:

|                     |          |
|---------------------|----------|
| Rent Invoice        | +₹20,000 |
| Electricity Invoice | +₹1,400  |
| Payment             | -₹14,000 |

|              |          |
|--------------|----------|
| Cash Payment | - ₹7,400 |
| Balance      | ₹0       |

## 29. Payment Corrections and Reversals

Finalized payments cannot be directly edited or deleted.

If an incorrect transaction is recorded, a reversal must be created.

Example:

Incorrect:

|                  |
|------------------|
| Payment +₹10,000 |
|------------------|

Correction:

|                  |          |
|------------------|----------|
| Original Payment | +₹10,000 |
| Reversal         | -₹10,000 |
| Correct Payment  | +₹1,000  |

The reversal should contain:

- Original transaction ID
- Reversal amount
- Reason
- Created by
- Created date

Complete history must be preserved.

## 30. Receipts

Receipts should be generated for successful payments.

Receipt should include:

- PG name
- Resident name

- Room/bed
- Receipt number
- Payment amount
- Payment mode
- Payment date
- Billing period
- Outstanding amount
- PG contact details

Resident can:

- View receipt
- Download receipt

Manager can also provide/send the receipt to resident.

---

## 31. Rent Reminders

Rent reminders should be generated five days before the resident's due date.

Notification recipients:

- Resident
- Relevant manager/admin

Example:

Your rent of ₹10,000 will be due on 17 October.

No automatic late-payment charge is required in V1.

---

## 32. Electricity Billing

The application must support different electricity billing methods.

### Method A: Room Meter

Example:

Previous Reading: 1240  
Current Reading: 1380

Consumption: 140 units

Rate: ₹8

Bill = ₹1,120

Room bill is distributed among applicable residents.

---

## Method B: Floor Meter

Electricity is calculated for the entire floor.

The resident pays only for days during which they actually lived in the PG.

Calculation should use resident-days.

Example:

Resident A:

30 days

Resident B:

15 days

Resident C:

10 days

Electricity should be proportionally distributed according to stay duration.

Residents who move in halfway through the billing period should not pay for days before their move-in.

---

## Method C: Manual Charge

Manager can manually enter electricity amounts where calculation is performed outside the system.

---

## 33. Electricity Configuration

Each PG can configure:

- Electricity method
- Unit rate
- Meter level
- Billing period
- Calculation method

Electricity rate can differ between PG properties.

---

## 34. Food Configuration

Each PG can configure one of the following:

Food Included in Rent  
Food as Paid Add-On  
No Food Service

---

## 35. Food Menu

Manager can publish weekly food menus.

Example:

| Day     | Breakfast    | Lunch      | Dinner      |
|---------|--------------|------------|-------------|
| Monday  | Aloo Paratha | Rajma Rice | Dal Roti    |
| Tuesday | Poha         | Chole Rice | Paneer Roti |

Residents can view the menu.

---

## 36. Food Poll

Residents can vote for the next day's food.

Voting occurs during the previous afternoon.

Poll options may include:

Paneer  
Dal  
Skip Meal

or:

Eating  
Not Eating

The exact poll structure should be configurable.

Manager can view poll results.

Owner does not need access to food voting results.

---

## 37. Complaint Management

Residents can submit complaints.

Complaint categories may include:

- Electricity
- Plumbing
- Cleaning
- Food
- Wi-Fi
- Furniture
- Security
- Other

Resident can provide:

- Complaint title
- Description
- Image
- Category

Complaint workflow:

Open  
↓  
In Progress  
↓  
Resolved

Residents can view their complaint status.

---

## 38. Notices

Manager/admin can publish notices.

Examples:

- Water supply interruption
- Maintenance
- Food menu update
- PG rules
- Rent notification
- Cleaning schedule

Notices can target:

- Entire PG
  - Floor
  - Specific residents
- 

## 39. Application Notifications

V1 will use in-app notifications.

Notifications may include:

- Rent due
- Payment received
- Receipt generated
- Electricity bill generated
- Complaint status update
- New notice
- Room transfer
- Checkout reminder
- KYC/document reminder

Push/WhatsApp/SMS integrations are not required for V1.

---

## 40. Checkout Process

Checkout should follow a structured process.

Example:

```
Notice Submitted
↓
Checkout Date
↓
Final Rent
↓
Final Electricity
↓
Other Charges
↓
Damage Charges
↓
Security Deposit Adjustment
↓
Refund Calculation
↓
Final Settlement
↓
Checkout Complete
```

---

## 41. Final Settlement

Example:

|                     |         |
|---------------------|---------|
| Security Deposit    | ₹10,000 |
| Pending Electricity | -₹1,200 |
| Damage Charge       | -₹500   |
| Refund Amount       | ₹8,300  |

The settlement should record:

- Security deposit balance
- Pending rent
- Electricity
- Add-ons
- Damage charges
- Other charges
- Deductions
- Refund amount
- Settlement date
- Settled by

---

## 42. Bed Release

A bed becomes available only after checkout is completed.

Flow:

```
graph TD;
  A[Occupied] --> B[Checkout Processing];
  B --> C[Settlement Complete];
  C --> D[Available];
```

---

## 43. Resident Timeline

The application should maintain an activity timeline.

Example:

```
10 Sep
Reservation created

10 Sep
₹5,000 security deposit received

15 Sep
```

Moved into Room 101-B

15 Sep  
Rent started at ₹10,000

10 Oct  
Rent reminder generated

15 Oct  
₹7,000 UPI payment received

16 Oct  
₹3,000 Cash received

20 Oct  
Complaint raised

21 Oct  
Complaint resolved

This improves auditability and dispute handling.

## 44. Admin Dashboard

The Owner dashboard should provide a high-level business overview.

Example:

|                       |        |
|-----------------------|--------|
| Total PGs             | 4      |
| Total Rooms           | 85     |
| Total Beds            | 240    |
| Occupied Beds         | 192    |
| Reserved Beds         | 12     |
| Available Beds        | 36     |
| Expected Rent         | ₹18.2L |
| Rent Collected        | ₹15.8L |
| Outstanding           | ₹2.4L  |
| Security Deposit Held | ₹21L   |
| Open Complaints       | 14     |

|                    |   |
|--------------------|---|
| Upcoming Move-Outs | 6 |
| New Reservations   | 9 |

Owner should be able to filter dashboard by:

- All PGs
- Individual PG
- Month
- Date range

## 45. Manager Dashboard

Manager dashboard should focus on operations.

Example:

|                     |    |
|---------------------|----|
| Occupied Beds       | 88 |
| Available Beds      | 12 |
| Reserved Beds       | 4  |
| Rent Due Today      | 5  |
| Rent Due This Week  | 13 |
| Pending Electricity | 8  |
| Open Complaints     | 6  |
| Today's Move-Ins    | 2  |
| Upcoming Move-Outs  | 3  |
| Tomorrow Food Poll  |    |
| Pending Votes       | 11 |

## 46. Resident Dashboard

Resident dashboard should remain simple.

Main cards:

My Room  
Room 101  
Bed 101-B

Next Rent  
₹10,000  
Due 17 October

Outstanding Due  
₹750 Electricity

Security Deposit  
₹10,000

Open Complaints  
1

Resident navigation:

- Home
- My Stay
- Payments
- Bills
- Receipts
- Food
- Complaints
- Notices
- Documents
- Profile

---

## 47. Reports

V1 should support basic reports.

### Occupancy Reports

- Occupied beds
- Vacant beds
- Reserved beds
- Room occupancy percentage

### Financial Reports

- Expected rent

- Collected rent
- Outstanding rent
- Electricity collection
- Add-on charges
- Security deposits

## **Resident Reports**

- Current residents
- Previous residents
- Upcoming checkout
- New move-ins
- Resident history

## **Complaint Reports**

- Open complaints
- Resolved complaints
- Complaints by category

---

# **48. Audit Trail**

Important operations must maintain audit history.

Examples:

- Rent revision
- Payment entry
- Payment reversal
- Security deposit change
- Room transfer
- PG transfer
- Electricity override
- Prorated rent override
- Checkout settlement

Audit record should capture:

- User
  - Action
  - Date/time
  - Previous value where applicable
  - New value
  - Reason where required
-

## 49. Search

Admin/manager should be able to search residents by:

- Name
- Mobile
- Room
- Bed
- PG
- Document reference where permitted

Search should include active and previous residents.

---

## 50. Security Requirements

The application will contain personally identifiable and financial information.

Basic security requirements include:

- Authentication required
  - Role-based authorization
  - Tenant data isolation
  - Secure password storage
  - HTTPS
  - Protected document access
  - Restricted financial actions
  - Audit logging
  - No permanent deletion of financial transactions
  - Sensitive document access restrictions
- 

## 51. Non-Functional Requirements

### Performance

Common screens should load quickly under normal PG usage.

Lists should support pagination.

---

## Scalability

Architecture should support:

- Multiple organizations
  - Multiple PGs per organization
  - Thousands of residents
  - Growing financial history
- 

## Responsiveness

Application should work on:

- Desktop
- Tablet
- Mobile browser

Resident experience should be especially mobile-friendly.

---

## Reliability

Financial transactions must maintain consistent history.

Failed operations must not result in partially saved financial data.

---

## 52. V1 Scope

V1 should include:

1. Authentication
2. Owner organization
3. Multi-PG management
4. Floor management
5. Room management
6. Bed management
7. Resident management
8. KYC/documents
9. Reservation
10. Security deposit
11. Move-in
12. Resident stays
13. Rent configuration

14. Individual billing cycle
  15. Rent revision
  16. Add-on services
  17. Monthly billing
  18. Payments
  19. Payment reversals
  20. Receipts
  21. Electricity billing
  22. Food menu
  23. Food poll
  24. Complaints
  25. Notices
  26. Notifications
  27. Room transfer
  28. PG transfer
  29. Notice period
  30. Lock-in
  31. Checkout
  32. Settlement
  33. Dashboards
  34. Basic reports
  35. Audit history
- 

## 53. Features Not Required in V1

The following should be kept outside V1:

- Online payment gateway
- WhatsApp integration
- SMS integration
- Biometric attendance
- Smart locks
- Visitor management
- Staff payroll
- Grocery inventory
- Kitchen inventory
- Full accounting
- GST accounting
- AI chatbot
- Facial recognition
- IoT electricity meters
- Automatic meter integrations
- Advanced CRM
- Marketplace/listing PG discovery

These can be considered after the core platform is stable.

---

## 54. Future SaaS Features

Potential future features include:

### Subscription Management

PG businesses could subscribe to plans such as:

Starter  
Growth  
Professional  
Enterprise

Plans may be based on:

- Number of PGs
- Number of beds
- Number of managers
- Storage
- Advanced reports

---

### Online Payments

Residents could eventually pay directly through:

- UPI
- Cards
- Net banking

The payment gateway can automatically reconcile invoices.

---

### WhatsApp Automation

Possible notifications:

- Rent reminder
- Receipt
- Electricity bill
- Notice
- Checkout reminder

---

## Visitor Management

Track:

- Visitor name
- Resident being visited
- Entry time
- Exit time

---

## Inventory

PG inventory such as:

- Beds
- Mattresses
- Fans
- AC units
- Furniture
- Appliances

---

## Maintenance Management

Track assets and recurring maintenance.

---

# 55. Primary Business Rules Summary

1. One owner can operate multiple PGs.
2. Every PG may have different floors, rooms and configurations.
3. Rooms can support single, double, triple, four-sharing and custom types.
4. Beds are individually identifiable.
5. Residents reserve specific beds.
6. Rent starts only from actual move-in date.
7. Reservations are manually cancelled.

8. Security deposits are tracked separately from rent.
9. Security deposits can be paid partially.
10. Reservation/security refund policy is configurable by PG.
11. Every resident has an individual rent cycle.
12. Month-end billing uses the last valid calendar date where required.
13. Rent can change with future effective dates.
14. Previous rent values must remain historically available.
15. Resident add-ons are independent from base rent.
16. Payments may use multiple payment modes.
17. Partial payments are allowed.
18. Finalized payments cannot be edited/deleted.
19. Corrections must use reversal transactions.
20. Electricity can be calculated by room meter, floor meter or manual entry.
21. Residents only pay floor electricity from the date they start living there.
22. Resident-day based allocation should be supported.
23. Food can be included, paid add-on or unavailable.
24. Food polling is managed by the manager.
25. Residents can raise complaints.
26. Every PG can define its own lock-in period.
27. Every PG can define its own notice period.
28. Manager can override certain rent/transfer calculations where allowed.
29. Resident transfers must preserve history.
30. Checkout requires final financial settlement.

31. Financial history must remain auditable.

---

## 56. Product Success Criteria

The initial product should be considered successful when a PG can completely operate its core resident-management process without relying on Excel for daily tracking.

A manager should be able to:

- Identify every occupied and available bed
- Find a resident instantly
- Know each resident's rent
- Know every outstanding payment
- Track security deposits
- Generate receipts
- Calculate electricity
- Handle complaints
- Manage food voting
- Transfer residents
- Complete checkout
- View historical financial records

The Owner should be able to understand the state of all their PG properties from one dashboard.

The resident should be able to understand exactly:

- Where they are staying
  - What they owe
  - What they have paid
  - Their receipts
  - Their electricity charges
  - Their services
  - Their complaints
  - Their PG notices
- 

## 57. Recommended V1 Development Order

The recommended implementation sequence is:

Phase 1  
Authentication + SaaS Tenant Structure

Phase 2

PG → Floor → Room → Bed

Phase 3

Resident + KYC + Documents

Phase 4

Reservation + Move-In + Resident Stay

Phase 5

Rent + Rent Cycles + Add-Ons

Phase 6

Invoices + Payments + Ledger + Receipts

Phase 7

Security Deposit

Phase 8

Electricity

Phase 9

Room/PG Transfers

Phase 10

Notice + Checkout + Settlement

Phase 11

Complaints + Notices

Phase 12

Food Menu + Poll

Phase 13

Dashboards + Reports

Phase 14

Audit + Security Hardening + SaaS Administration

This order avoids building UI modules before the underlying resident and financial models are stable.

---

## 58. Recommended Next Technical Deliverables

The PRD should next be converted into:

## 1. System Architecture

Define:

- Frontend
- Backend
- Database
- Authentication
- SaaS tenant strategy
- File storage
- Notification architecture

## 2. Database ERD

Define tables and relationships for:

```
Organization
PG
Floor
Room
Bed
Resident
ResidentStay
Reservation
ResidentDocument
RentHistory
ResidentService
Invoice
InvoiceItem
Payment
PaymentAllocation
TransactionReversal
SecurityDeposit
ElectricityBill
Complaint
FoodMenu
FoodPoll
Notice
Notification
Checkout
AuditLog
```

## 3. API Specification

Example modules:

```
/api/pg  
/api/floors  
/api/rooms  
/api/beds  
/api/residents  
/api/reservations  
/api/stays  
/api/invoices  
/api/payments  
/api/electricity  
/api/complaints  
/api/food  
/api/checkout
```

#### **4. UI Screen Map**

Define every Admin, Manager and Resident screen.

#### **5. Development Backlog**

Convert each requirement into:

```
Epic  
↓  
Feature  
↓  
User Story  
↓  
API  
↓  
UI  
↓  
Acceptance Criteria
```

These documents should be created before significant application development begins.